Linux网络--IO多路转接层02 poll原理与select服务器改写

> ⓘ 系列导航
>
> 上一篇：Linux网络--IO多路转接层01 从阻塞IO到select事件循环
> 系列索引：Linux网络--IO多路转接与Reactor系列总览
> 下一篇：Linux网络--IO多路转接层03 epoll事件模型与内核原理

poll 原理与 select 服务器改写

1. poll 要解决的具体问题

select 把“关心什么”和“什么就绪”放在同一组位图中，导致调用之后原集合被修改。应用每轮都要重新构造位图、重新计算最大 fd，再把整个集合交给内核。

poll 仍然让用户传入一组等待对象，但把数据拆成三部分：

```
#include <poll.h>

struct pollfd {
    int   fd;       // 监视哪个 fd
    short events;   // 用户关心什么
    short revents;  // 内核报告什么
};

int poll(struct pollfd *fds, nfds_t nfds, int timeout);
```

`events` 是输入，`revents` 是输出。两者分离之后，下一轮调用可以保留 `events`，不需要因为内核写过结果而重建关心条件。

2. 参数与返回值

2.1 fds 和 nfds

`fds` 是结构体数组首地址，`nfds` 是数组元素数量。数组可以是固定长度，也可以动态扩展。

未使用的槽位约定把 `fd` 设置为 `-1`。内核会忽略这样的元素。

2.2 timeout

单位是毫秒：

- `-1`：永久阻塞；
- `0`：立即返回；
- 正数：最多等待指定毫秒。

2.3 返回值

- `ret > 0`：有 `ret` 个元素的 `revents` 非零；
- `ret == 0`：超时；
- `ret < 0`：错误。

一个 fd 可以同时出现多个事件，例如同时可读并且挂断，但它仍然只算一个就绪元素。

3. 常用事件

事件	含义
`POLLIN`	普通数据或优先数据可读
`POLLOUT`	可以写
`POLLERR`	发生错误
`POLLHUP`	挂断
`POLLRDHUP`	对端关闭写方向，Linux 扩展

事件是位标志，设置和检查都使用按位运算：

```
fds[i].events = POLLIN | POLLOUT;

if (fds[i].revents & POLLIN) {
    // 可读
}

if (fds[i].revents & (POLLERR | POLLHUP)) {
```

```
    // 错误或挂断
}
```

socket 的就绪条件和 select 相同：读缓冲区达到低水位、收到 FIN、监听队列有新连接、写缓冲区有空间等。poll 改变的是表达方式，不是内核判断“可读/可写”的根本规则。

4. 从 select 到 poll 的映射

select	poll
`readfds`	`events = POLLIN`
`writefds`	`events = POLLOUT`
调用后位图	每个元素的 `revents`
`FD_SET`	设置 `events`
`FD_ISSET`	`revents & event`
`/ 0` 表示空位	`fd = -1` 表示空位

select 的三个位图被替换成一个 `pollfd` 数组。fd 本身、关心的事件和返回的事件集中在一个逻辑单元中，管理连接时更直观。

5. PollServer 的整体结构

改写后的服务器可以保留原来“监听 socket + 普通连接”的事件分支，只替换等待层。

```
class PollServer {
private:
    std::unique_ptr<TcpSocket> listen_sock_;
    struct pollfd* fds_;
    int fd_count_;
    int port_;
    bool running_;
};
```

5.1 初始化数组

```
void InitServer() {
    listen_sock_ = std::make_unique<TcpSocket>();
```

```
    listen_sock_->BuildListenSocket(port_, backlog);

    fds_ = new pollfd[fd_count_];
    for (int i = 0; i < fd_count_; ++i) {
        fds_[i].fd = -1;
        fds_[i].events = 0;
        fds_[i].revents = 0;
    }

    int listen_fd = listen_sock_->GetFd();
    fds_[0].fd = listen_fd;
    fds_[0].events = POLLIN;
}
```

初始化时把所有槽位置为未使用，再把监听 fd 放入第一个槽位。之后每次 `poll` 都会同时等待监听 socket 和所有普通连接。

5.2 主循环

```
void Loop() {
    running_ = true;

    while (running_) {
        int n = poll(fds_, fd_count_, -1);

        if (n < 0) {
            if (errno == EINTR) continue;
            log_error("poll failed");
            break;
        }
        if (n == 0) {
            continue;
        }

        HandleEvents();
    }
}
```

和 select 相比，这里不需要在每次调用前重建 `events`。`revents` 是上一次输出，但本次 `poll` 会重新填写有效元素。

5.3 处理所有返回事件

```
void HandleEvents() {
    for (int i = 0; i < fd_count_; ++i) {
        int fd = fds_[i].fd;
        short revents = fds_[i].revents;

        if (fd == -1 || revents == 0) continue;

        if (revents & POLLIN) {
            if (fd == listen_sock_->GetFd()) {
                HandleAccept();
            } else {
                HandleRead(fd, i);
            }
        }

        if (revents & (POLLERR | POLLHUP)) {
            CloseAt(i);
        }
    }
}
```

poll 返回后仍要遍历整个数组，因为接口不告诉你就绪项位于哪些下标。它只保证 `revents` 的内容可用于筛选。

6. 接收新连接

监听 fd 可读后，`accept` 不应该因为“没有数据到来”而阻塞，因为监听 socket 已经报告存在待处理连接。

```
void HandleAccept() {
    TcpSocket client;
    std::string ip;
    uint16_t port;

    int fd = listen_sock_->AcceptConnection(&client, &ip, &port);
    if (fd < 0) {
        if (errno == EINTR) return;
        log_error("accept failed");
        return;
    }

    int slot = FindEmptySlot();
    if (slot == -1) {
```

```
        client.Close();
        log_warning("connection table is full");
        return;
    }

    fds_[slot].fd = fd;
    fds_[slot].events = POLLIN;
    fds_[slot].revents = 0;
}

int FindEmptySlot() {
    for (int i = 0; i < fd_count_; ++i) {
        if (fds_[i].fd == -1) return i;
    }
    return -1;
}
```

新连接不能立刻假设“已经可以读业务数据”。把它放进 poll 表，等下一次 `POLLIN` 后再读，才保持事件驱动模型的完整逻辑。

如果数组已满，可以选择扩容、暂时拒绝新连接或用队列等待空位。直接关闭连接并记录告警是最简单的一种保护策略。

7. 处理普通连接的读事件

```
void HandleRead(int fd, int index) {
    char buffer[1024];

    ssize_t n = recv(fd, buffer, sizeof(buffer), 0);

    if (n > 0) {
        std::string request(buffer, n);
        std::string response;
        handler(request, &response);

        size_t sent = 0;
        while (sent < response.size()) {
            ssize_t w = send(fd, response.data() + sent,
                             response.size() - sent, MSG_NOSIGNAL);
            if (w > 0) {
                sent += static_cast<size_t>(w);
                continue;
            }
```

```
            if (w < 0 && errno == EINTR) continue;
            if (w < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
                // 教学版简化为关闭连接；完整 Reactor 会保存剩余数据
                CloseAt(index);
                return;
            }
            CloseAt(index);
            return;
        }
        return;
    }

    if (n == 0) {
        // 对端关闭
        CloseAt(index);
        return;
    }

    if (errno == EINTR) return;
    if (errno == EAGAIN || errno == EWOULDBLOCK) return;

    CloseAt(index);
}
```

示例为了突出 poll 事件循环，把发送简化为一次完成。真实服务如果允许发送缓冲区暂时不足，必须保留剩余响应，并按需关心 `POLLOUT`。这个改进会在 Reactor 的 Connection 输出缓冲区中正式实现。

8. 关闭连接并释放槽位

```
void CloseAt(int index) {
    if (fds_[index].fd == -1) return;

    close(fds_[index].fd);
    fds_[index].fd = -1;
    fds_[index].events = 0;
    fds_[index].revents = 0;
}
```

关键状态变化：

```
有效连接：fd >= 0，events = POLLIN
        |
        v
recv 返回 0 或出错
        |
        v
close(fd)
        |
        v
空槽位：fd = -1，events = 0
```

槽位重置是 poll 服务器的核心生命周期规则。如果不把 `fd` 设为 `-1`，下一次 poll 可能继续监听已经关闭或已经被复用的编号。

9. poll 改进了什么

9.1 输入输出解耦

`events` 长期表示关心的事件，`revents` 只表示本次返回结果。用户不再需要像 select 那样保护一份原集合。

9.2 接口没有固定位图上限

数组长度由用户决定，可以表达比常见 `FD_SETSIZE` 更多的 fd。但 fd 总数仍受进程资源和内存限制，“接口没有固定上限”不能推出“系统可以无限连接”。

9.3 事件表达更细

可以直接使用 `POLLRDHUP` 等扩展事件，处理对端关闭读写方向更加直观。

10. poll 没有解决的问题

poll 的底层等待仍需要让内核检查传入的 fd 数组。连接数很多而活跃连接很少时：

1. 整个 `pollfd` 数组仍要复制到内核；
2. 内核仍要检查大量未活跃 fd；
3. 返回后用户仍要扫描整个数组，寻找 `revents != 0` 的元素。

所以它改进了 select 的数据表达和重复设置问题，却没有改变“等待集合由用户每次提交、内核返回后再由用户全量筛选”的根本结构。

11. poll 与 select 的工程取舍

场景	更合适的选择
需要兼容性极广的小型程序	select
连接数中等、希望事件字段单一清楚	poll
需要监听大量长连接且活跃比例低	epoll
连接极少但持续活跃	三者差距可能不明显，应以测量为准

poll 适合教学递进，也适合不少普通服务。它比 select 更容易维护，但没有必要把它说成 epoll 的“低配版”：两者的接口模型仍是“每轮提交等待集合”，epoll 才把关注集合下沉为内核持久状态。

12. 过渡到 epoll

poll 已经能清楚地表示每个 fd 关心什么，但每次调用仍需要提交完整数组。如果内核能在第一次注册时保存关心关系，之后只把发生变化的项放回用户，就能进一步减少重复工作。

这正是 epoll 的核心变化：

```
select / poll：
    每轮提交关注集合 -> 内核检查全部 -> 用户筛选结果

epoll：
    注册时保存关注集合 -> 就绪时进入队列 -> 只返回就绪项
```

下一篇从 epoll 的三个系统调用开始，进一步解释红黑树、就绪队列和内核回调怎样配合。